

Object / Blob Storage Basics

Phase 1 · Fundamentals · Estimated time: 1 hour

1. Concept From First Principles

Not everything belongs in a database. A profile photo, a PDF invoice, a video, a CSV export — these are large, unstructured **files**, and stuffing them directly into a relational database (as a BLOB column) or even your application server's local disk causes real problems at scale: databases become bloated and slow, and a file saved to one server's local disk simply isn't visible to the other 49 servers behind your load balancer.

Object storage (AWS S3, Google Cloud Storage, Azure Blob Storage) solves this: it's a separate, purpose-built service for storing and retrieving files ("objects") over a simple HTTP API, completely decoupled from your application servers and your database.

Backend engineer analogy

If you've ever used Laravel's `Storage::disk('s3')->put(...)` instead of saving an uploaded file to the local `storage/app/public` folder, you've already made this exact HLD decision — and you made it for exactly the reason system design interviews expect: local disk storage doesn't survive a server restart, isn't shared across multiple app servers, and doesn't scale.

One more property that matters for HLD reasoning: object storage is typically **durable by design** — providers like S3 replicate every object across multiple physical facilities automatically, advertising extremely high durability guarantees (often eleven nines). This means, unlike a single database server's local disk, you generally don't need to design your own replication scheme just to avoid losing an uploaded file — that problem is already solved for you at the storage layer, which is part of why object storage is such a low-risk, high-leverage default for anything file-shaped.

Object storage vs. file storage vs. block storage

- **Object storage:** flat structure, each file is an "object" with a unique key and metadata, accessed over HTTP. No folder hierarchy in the traditional sense (though keys can look like paths, e.g., `invoices/2026/08/inv-8821.pdf`). Extremely scalable, but not designed for frequent small edits to a file.
- **File storage:** traditional hierarchical folders/directories (like a shared network drive, e.g., NFS). Good when an application genuinely needs POSIX-style file operations.
- **Block storage:** raw storage volumes attached to a single server (like an EBS volume) — what a database's underlying disk usually sits on. Fast, but tied to one machine unless specifically designed for sharing.

The rule of thumb for HLD interviews: if it's a large, mostly-immutable file that needs to be durably stored and served to many clients (images, videos, documents, backups, exports), object storage is almost always the right default answer.

A pattern worth naming explicitly: for large uploads, well-designed systems don't route the file's bytes through the application server at all. Instead, the app server generates a short-lived, permission-scoped **pre-signed URL** and hands it to the client, which then uploads (or downloads) directly to/from object storage. This avoids burning app-server bandwidth and memory on large file transfers entirely — the app server's only job is authorization ("yes, this user may upload here"), not moving the actual bytes.

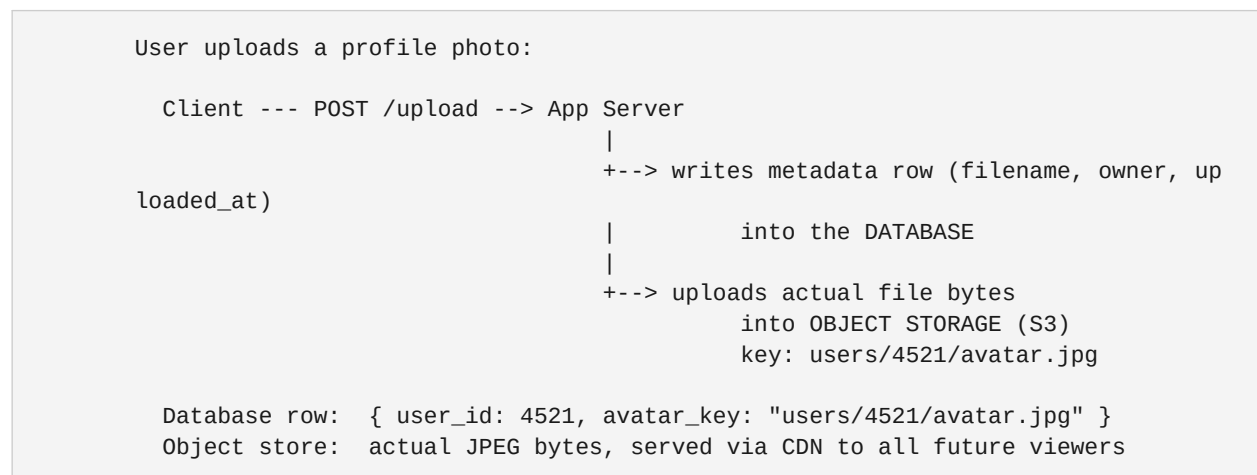
2. Real-World Example / Case Study

When a Zomato/Swiggy-style restaurant partner uploads menu photos, or a Flipkart seller uploads product images, those files go to object storage (S3 or equivalent), not the application database. The database only stores a *reference* — a URL or object key pointing at the file — while the actual bytes live in object storage, often served to end users through a CDN (Chapter 10) sitting in front of it for speed.

Razorpay and PhonePe store transaction receipts, KYC documents, and settlement reports in object storage with strict access controls and encryption at rest — a pattern you'll see repeated in Phase 3's security chapter. The database holds structured records (transaction ID, amount, status); object storage holds the unstructured supporting documents.

A third example: Talabat's menu-photo pipeline likely uses pre-signed upload URLs for restaurant partners uploading dish photos directly from their phones, followed by an asynchronous image-processing step (resizing, compression, generating thumbnails) triggered by an event once the upload lands in object storage — combining this chapter's object storage pattern with Chapter 5's async processing and a preview of the event-driven patterns Phase 3 covers in depth.

3. Diagram



4. Trade-offs Table

Storage type	Best for	Weak for
Object storage (S3-style)	Images, videos, documents, backups, exports	Frequent small in-place edits, low-latency random access
File storage (NFS-style)	Shared app config, legacy apps needing POSIX file ops	Massive horizontal scale, geographic distribution

Storage type	Best for	Weak for
Block storage (EBS-style)	Database disks, single-server high-performance storage	Sharing the same volume across many servers directly
Database BLOB column	Tiny files tightly coupled to a row's lifecycle	Anything large or frequently accessed — bloats the DB

5. Common Interview Questions

Q1: "Where would you store user-uploaded profile photos, and why not just the database?"

Model approach: Recommend object storage, explain that storing large binary files in a relational DB bloats table size, slows backups, and doesn't scale well for high-volume reads, then describe storing only a reference/URL in the database while the file itself lives in S3-equivalent storage, typically fronted by a CDN for fast delivery.

Q2: "How would you design storage for a video-upload feature?"

Model approach: Object storage for the raw and processed video files, a database for video metadata (title, owner, status, duration), and mention that video processing (transcoding to multiple resolutions) typically happens asynchronously via a queue (Chapter 5 sync/async, expanded in Phase 2) after upload, not inline with the upload request.

Q3: "Why route a large file upload directly to object storage instead of through your application server?"

Model approach: Explain that streaming large files through the app server wastes its bandwidth, memory, and connection slots on pure data transfer instead of actual application logic, and doesn't scale well with concurrent large uploads. A pre-signed URL lets the app server just authorize the transfer, while the client talks directly to object storage, which is purpose-built to handle massive concurrent transfer volume.

6. Practice Exercises

- 1 For a food delivery app, list 3 things that belong in object storage and 3 that belong in a relational database, with reasons.
- 2 Explain why saving an uploaded file to an app server's local disk breaks once you have more than one app server.
- 3 Sketch the data flow (from this chapter's diagram) for a PDF invoice generation feature instead of a photo upload.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"AWS S3 Explained Object Storage Service for Files, Buckets & Cloud Data"	Direct, focused explanation of object storage mechanics.
English	"Amazon Simple Storage Service (S3): Overview and Tutorial"	Practical overview of how S3-style storage is used in real systems.

Language	Video & Channel	Why it helps
Hindi	"Amazon S3 Tutorial for Beginners in Hindi" — Great Learning	Hindi walkthrough covering S3 basics, storage classes, and use cases.
Hindi	"Complete System Design Roadmap 2025 HLD & LLD" — CodeHelp	Places object storage in the context of a broader Hindi system design roadmap.

8. Key Takeaways

- Large, unstructured files (images, videos, documents) belong in object storage, not the database.
- Object storage is accessed over HTTP as key-value-like "objects," independent of your app servers.
- The database should store only a reference (URL/key) to the file, not the file's bytes.
- Local disk storage on an app server breaks the moment you have more than one server instance.
- Object storage is typically fronted by a CDN for fast, cheap delivery to end users.
- Block storage and file storage are different tools for different jobs — know when each applies.